

Design by Contract

Bertrand Meyer — Interactive Software Engineering Inc.

In: Advances in Object-Oriented Software Engineering, Prentice Hall, 1991.

1. Apresentação e Contexto

O capítulo Design by Contract, de Bertrand Meyer, é uma das contribuições mais influentes da engenharia de software orientada a objetos. Publicado no início dos anos 1990 e fortemente ligado à linguagem Eiffel, o texto propõe uma metáfora simples e poderosa: a construção de software como uma série de decisões contratuais documentadas entre módulos clientes e fornecedores. O objetivo central é elevar a confiabilidade de sistemas por meio de especificações formais embutidas diretamente no código — não como comentários, mas como asserções verificáveis em tempo de execução.

Meyer parte de uma crítica legítima ao estado da prática da época: apesar de décadas de pesquisa em verificação formal e especificação, esses avanços raramente chegavam ao desenvolvimento cotidiano. A literatura sobre orientação a objetos, em particular, priorizava flexibilidade e modularidade enquanto tratava correção e robustez como preocupações secundárias — um paradoxo grave numa abordagem centrada em reuso.

2. Ideias Centrais

O coração do artigo é o mecanismo de asserções: **pré-condições**, **pós-condições** e **invariantes de classe**. A pré-condição define o que o cliente deve garantir antes de chamar uma rotina; a pós-condição, o que o fornecedor garante após a execução; e o invariante descreve propriedades que todo objeto de uma classe deve satisfazer em qualquer estado observável. Juntos, esses três elementos compõem um contrato formal e verificável, eliminando a ambiguidade nas interfaces.

Um ponto particularmente relevante é a crítica à programação defensiva. Meyer argumenta que verificar redundantemente as mesmas condições tanto no cliente quanto no fornecedor não aumenta a confiabilidade — pelo contrário, incha o código com verificações sobrepostas e obscurece a lógica essencial. O contrato resolve isso ao atribuir responsabilidade clara: ou a condição pertence à pré-condição (ônus do cliente), ou pertence ao corpo da rotina (ônus do fornecedor) — nunca ambos.

O artigo também desenvolve uma teoria disciplinada de tratamento de exceções, derivada diretamente do modelo contratual. Meyer propõe três leis: (1) uma chamada termina ou cumprindo o contrato, ou falhando — sem fingir sucesso; (2) a falha de uma rotina deve sempre gerar uma exceção no chamador; (3) ao tratar uma exceção, a rotina deve ou tentar novamente (resumption) ou encerrar em pânico organizado (organized panic), sempre restaurando o invariante. O mecanismo Eiffel, com apenas duas palavras-chave (**rescue** e **retry**), se mostra mais simples e correto do que as abordagens ad hoc de Ada e CLU.

Por fim, o autor aplica o mesmo raciocínio à herança: redefinir uma rotina em uma subclasse é, conceitualmente, subcontratar. A regra de redefinição exige pré-condição mais fraca (aceitar mais casos) e pós-condição mais forte (entregar mais) — o que fundamenta o Princípio de Substituição

de Liskov muito antes de sua formulação popular.

3. Aplicação Prática: API de Precificação do Hermes (NestJS + TypeORM) O Hermes é uma ferramenta SaaS para vendedores Amazon que, entre outras funcionalidades, expõe uma API de precificação — lógica sensível, pois erros afetam diretamente a margem de contribuição do lojista. Desenvolvida em NestJS com TypeScript e TypeORM, essa API ilustra de forma clara onde o Design by Contract agrega valor imediato.

Considere um **serviço de cálculo de preço sugerido**. Sem contratos explícitos, é comum que cada camada repita as mesmas guards: o controller valida se o custo é positivo, o service verifica novamente, e o repositório ainda faz um terceiro check. O resultado é exatamente o "oceano de código de verificação de erros" que Meyer critica — difícil de manter e com responsabilidade difusa.

Com DbC, o contrato seria expresso assim (pseudo-código TypeScript comentado):

```
// PRÉ-CONDIÇÃO (responsabilidade do caller):  
// cost > 0 && taxRate >= 0 && taxRate < 1 && targetMargin > 0  
// PÓS-CONDIÇÃO (garantia do serviço):  
// result.suggestedPrice >= cost / (1 - targetMargin)  
// result.netMargin >= targetMargin  
// INVARIANTE (PricingConfig):  
// 0 < baseMargin <= 1 && platformFee >= 0
```

Na prática, isso significa que a camada de controller continua validando inputs externos (fronteira não confiável), mas o PricingService pode assumir que seus argumentos já são válidos — sem if-guards redundantes. Se a pré-condição for violada, é um bug do caller, não do service. Se a pós-condição falhar, é um bug interno: a exceção deve propagar, não ser silenciada com um fallback opaco que retornaria um preço incorreto ao vendedor.

Outro ponto de aplicação direta é a **regra de redefinição na herança**. O Hermes possui diferentes estratégias de precificação (margem fixa, precificação dinâmica baseada em concorrência, preço mínimo garantido). Se essas estratégias forem modeladas como subclasses de um PricingStrategy base, a regra de Meyer exige que cada refinamento aceite ao menos os mesmos inputs do contrato pai e entregue ao menos a mesma garantia de margem — impedindo que uma estratégia derivada quebre silenciosamente o comportamento esperado pelos módulos que dependem da interface base.

4. Avaliação Crítica

O artigo tem enorme valor conceitual e permanece relevante. Suas ideias permeiam práticas modernas: anotações de validação (class-validator no NestJS), tipos não-anuláveis no TypeScript, testes baseados em propriedades (property-based testing) e o próprio Princípio de Substituição de Liskov são, em essência, manifestações da mesma filosofia contratual.

A principal limitação é a dependência de uma linguagem específica — Eiffel — e de um mecanismo de asserções nativo que linguagens como TypeScript não possuem de forma idiomática. Na prática, os contratos tendem a viver em documentação, testes ou bibliotecas de validação separadas do código de produção, o que enfraquece a garantia de que serão verificados. Ainda assim, a disciplina mental proposta pelo autor — quem é responsável por esta condição? — é um guia de design valioso independente da linguagem, e sua aplicação explícita reduziria

significativamente a quantidade de bugs silenciosos em APIs críticas como as do Hermes.

Resenha elaborada para a disciplina de Arquitetura de Software — PUC Minas, 2026.